

# Real-Time Linux Networking and Troubleshooting Commands

## Objective:

To gain hands-on experience using Linux commands for network configuration, troubleshooting, and system monitoring by simulating real-world industry scenarios.

---

## Industry Scenario:

You are working as a **Junior DevOps Engineer** at a **SaaS company** called **TechCloud Inc.** One of your production web servers (Ubuntu-based) is experiencing **intermittent connectivity issues and slow response times**.

Your **task** is to investigate the issue using essential Linux commands, document your findings, and report actionable insights.

## Prerequisites:

- Running EC2 instance (Ubuntu)
  - Create IAM user
    - Give EC2 full permissions
    - Access EC2 via IAM user
- 

## Task Instructions

### ♦ Task 1: Check IP and Connectivity

1. **Command:** `ip a`
  - **Objective:** List all network interfaces and their IPs.
    - i. Displays all IP addresses and network interfaces on the system.
    - ii. Shows your computer's internet connection details.

### Expected Output Snippet:

```
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500  
    inet 192.168.0.101/24 brd 192.168.0.255 scope global dynamic  
ens33
```

○

#### 2. **Command:** `ping google.com`

- **Objective:** Test internet connectivity.
  - i. Tests network connectivity to Google by sending ICMP echo requests.
    - 1. ICMP (Internet Message Control Protocol) is protocol that helps troubleshoot and manage network problems
  - ii. Checks if your computer can reach Google.

### Expected Output:

```
PING google.com (142.250.190.14) 56(84) bytes of data.  
64 bytes from lax02s29-in-f14.1e100.net: icmp_seq=1 ttl=115  
time=18.5 ms
```

○

#### 3. **Command:** `curl ifconfig.me`

- **Objective:** Get the server's public IP.
  - i. Retrieves your public IP address using an external service.
  - ii. Tells you your internet (public) IP address.

### Expected Output:

```
203.0.113.45
```

○

---

## ♦ Task 2: Check Open Ports and Services

### 4. Command: `netstat -tulnp`

- **Objective:** List all open ports and running services.
  - i. Shows all listening ports and associated processes (TCP/UDP).
  - ii. Lists which programs are using the internet or network.

#### Expected Output:

```
tcp        0      0 0.0.0.0:80          0.0.0.0:*        LISTEN
1234/nginx: master
```

○

### 5. Command: `ss -tuln`

- **Objective:** Use modern replacement of netstat.
  - i. Displays listening TCP/UDP sockets without resolving names.
  - ii. Shows which apps are ready to connect to the internet.

#### Expected Output:

```
LISTEN 0      128      *:22             *:*
```

```
LISTEN 0      128      *:80             *:*
```

○

---

## ♦ Task 3: DNS and Routing

### 6. Command: `wget htcleaom/sample.txt`

- **Objective:** Test web download functionality.
  - i. Downloads a file from the specified URL.

1. [https://www.filesampleshub.com/format/document/txt/google\\_vignette](https://www.filesampleshub.com/format/document/txt/google_vignette)
- ii. Downloads a file from a website.

**Expected Output:**

```
Saving to: 'sample.txt'
sample.txt          100%[=====>]    127
--.-KB/s    in 0s
```

○

7. **Command:** `traceroute google.com`

- **Objective:** Diagnose routing path to Google.
  - i. Traces the path packets take to reach Google.
  - ii. Shows the path your internet traffic takes to reach Google.

**Expected Output:**

```
1  192.168.0.1  1.123 ms
2  10.10.0.1    3.456 ms
3  142.250.190.14 25.678 ms
```

○

8. **Command:** `ifconfig`

- **Objective:** (Deprecated) Still check interfaces (alternative to `ip a`).
  - i. Shows network interfaces and IP configuration (older tool)
  - ii. Shows internet and network settings (older method).

**Expected Output:**

```
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
```

```
inet 192.168.0.101 netmask 255.255.255.0 broadcast
192.168.0.255
```

○

9. **Command:** `nslookup google.com`

- **Objective:** Check DNS resolution.
  - i. Queries DNS to resolve Google's IP address.
  - ii. Finds the IP address of a website.

**Expected Output:**

```
Name:      google.com
Address: 142.250.190.14
```

○

---

#### ◆ Task 4: Troubleshoot and Audit Logs

10. **Command:** `grep "error" /var/log/syslog`

- **Objective:** Search for error logs.
  - i. Looks for errors in the system log.

**Expected Output:**

```
May 12 07:45:10 server1 nginx[1234]: [error] 500 Internal Server
Error
```

○

11. **Command:** `find /etc -name "nginx*"`

- **Objective:** Locate nginx-related configuration files.
  - i. Finds all files in `/etc` matching the name pattern "nginx\*".

### Expected Output:

```
/etc/nginx  
/etc/nginx/nginx.conf
```

○

### 12. Command: `history`

- **Objective:** View previously executed commands.

### Expected Output:

```
101 ip a  
102 ping google.com  
103 grep "error" /var/log/syslog
```

○

### 13. Command: `cut -d ':' -f1 /etc/passwd`

- **Objective:** List all usernames on the system.

### Expected Output:

```
root  
daemon  
syslog  
Ubuntu
```

## Task 5: Configure UFW Firewall

A **firewall** is like a **security guard** for your computer or server. It decides what is allowed in and what is blocked based on rules.

On a web server, we allow:

- ✓ Port **22** (SSH) for admins to manage the server
- ✓ Port **80** (HTTP) for people to view the website
- ✗ Block all other ports to protect from hackers

### [Commands to Execute]

```
sudo apt install ufw          # Install if not present
sudo ufw enable               # Enable firewall
sudo ufw allow ssh            # Allow SSH port (22)
sudo ufw allow http           # Allow HTTP port (80)

# View rules

sudo ufw status verbose
```

### Expected Output:

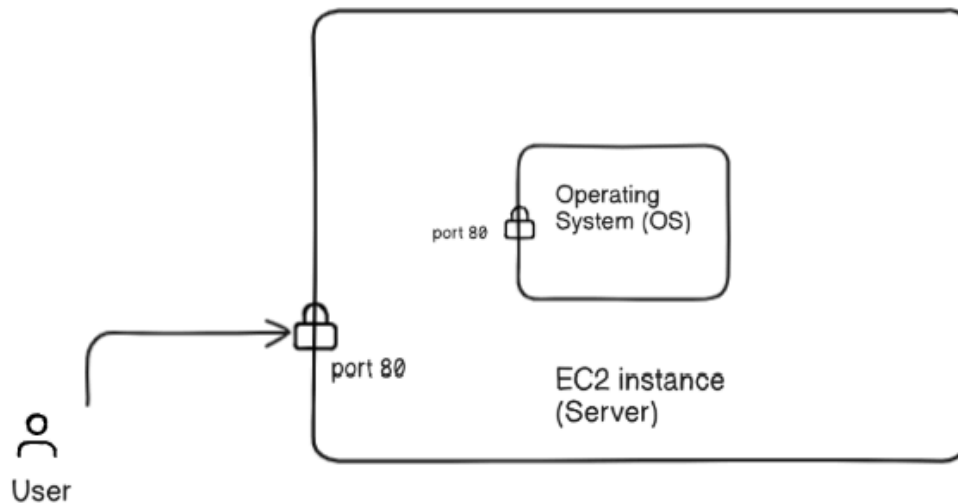
```
Status: active
```

| To | Action | From     |
|----|--------|----------|
| -- | -----  | ----     |
| 22 | ALLOW  | Anywhere |
| 80 | ALLOW  | Anywhere |

### Verify the firewall

```
sudo netstat -tuln | grep :80
```

## Firewall for Incoming Traffic



## Conclusion / Summary

This class gave you hands-on practice with essential Linux commands used daily by DevOps and system admins. You simulated a real-world scenario to troubleshoot network issues, check system logs, and verify connectivity and DNS.

---

### 🎓 Key Takeaways:

- ☒ Learned to use core networking and troubleshooting commands
- ☒ Practiced diagnosing real system issues
- ☒ Improved command-line and log analysis skills
- ☒ Secure a server using the UFW firewall and security groups

You're now better prepared to handle real production environments. Great job!